Q01: How Can Next-Token Prediction Lead to Problem Solving, Coding, and Instruction Following?

DLS 2026 — Draft module

October 2026

Learning objectives

After this module, a student should be able to (i) state the next-token objective precisely, (ii) explain why the objective can reward modeling structure in text and code, (iii) distinguish observed capability from a settled account of mechanism, and (iv) propose an evaluation that separates memorization, pattern completion, and generalization.

1 The narrow training objective

An autoregressive language model assigns a probability to each next token conditional on preceding tokens. Given a token sequence $x_{1:T}$, training commonly minimizes

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

The objective does *not* explicitly instruct the model to reason, write programs, follow an instruction, or represent a world model. It asks for a probability distribution that predicts continuations in the training distribution. A Transformer provides a flexible way to compute that distribution from the preceding sequence using attention and feed-forward layers [4].

This apparent mismatch is the source of the question. The useful first answer is not that next-token prediction secretly contains a reasoning rule. Rather, many continuations are hard to predict without tracking regularities that humans describe as syntax, facts, goals, causal relations, conventions, and intermediate steps. If a document says "the function returns the sum of the list," continuations in code, tests, and prose constrain which statements and tokens are likely next. A model that represents useful latent regularities can reduce prediction loss across many such contexts.

2 Why broad capabilities can be useful for prediction

Three routes help explain why the single objective can support varied behavior.

Compression of repeated structure. Natural language and code contain repeated patterns at several scales: spelling, grammar, APIs, algorithms, argument structures, and task formats. A representation that captures a reusable pattern can improve predictions in many places. This is an explanation of *why* representations may be useful; it is not proof that a particular internal feature carries a human-readable concept.

Learning from the prompt. At inference time the prompt is part of the model's context. Demonstrations, constraints, and a question can therefore alter the next-token distribution without changing model weights. Large language models showed substantial few-shot behavior under this setup [1]. In a controlled family of small Transformers, induction heads were associated with a sharp increase in an in-context learning behavior [2]. This is a concrete mechanistic clue, not a universal explanation of all in-context learning.

Training data contains tasks and solutions. Text and code corpora include explanations, debugging conversations, specifications, proofs, examples, and instruction-like exchanges. The model is rewarded for predicting the continuation of those patterns. Post-training can further alter which helpful, safe, or instruction-following continuation is preferred; Q01 concerns the pre-training objective and should not collapse this distinction.

3 What this does *not* establish

A fluent continuation is not itself evidence of reliable reasoning. The same model can produce a correct multi-step derivation, a plausible but false derivation, or a memorized fragment. Evaluations must therefore test held-out distributions, vary surface form, require verifiable intermediate artifacts where appropriate, and report failure modes.

Likewise, "emergence" is a description of an observed performance pattern, not a complete causal explanation. Some capabilities can look discontinuous when a continuous score is passed through a thresholded metric; this criticism is developed by Schaeffer et al. [3]. Other work documents sharp-looking changes on selected tasks as scale increases [5]. Students should ask: What metric was plotted? Is the effect robust to a different metric, model family, prompt, and contamination control?

4 A worked miniature: copying a latent rule

Consider the sequence `A B A B A`. Predicting its next token is easy if a model notices the repeating two-token pattern; predicting it by a fixed unigram frequency is not. The accompanying program implements two deliberately small predictors: a unigram baseline and an "induction-like" copy heuristic that finds an earlier occurrence of the current context and copies the token that followed it. The heuristic is not a Transformer and does not prove that real models work this way. It makes one key idea inspectable: a context-dependent rule can beat a context-free rule on a task whose continuation depends on latent structure.

5 Takeaway

Next-token prediction is a narrow objective applied to a broad, structured corpus. It can reward internal representations and context-sensitive algorithms that help predict language, code, and task demonstrations. That gives a plausible route to broad capabilities; it does not settle whether a response reflects robust reasoning, how a particular model implements it, or whether the behavior will generalize. Those are empirical questions requiring careful evaluation.

References

[1] Tom B. Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 2020.

[2] Catherine Olsson et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.

[3] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? *arXiv preprint arXiv:2304.15004*, 2023.

[4] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[5] Jason Wei et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.